

מס' שאלון - 530

6

ביולי 2020

מס' מועד 72

סמסטר 2020ב

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 4 שעות

בשאלון זה 12 עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

בהצלחה !!!

שאלה 1 (25 נקודות)

הרחיבו את שפת LET (שפת "ויהי") בביטוי חדש בשם switch

להלן הדקדוק המגדיר את התחביר החדש עבור ביטוי switch :

Expression ::= switch Expression {
{case Expression : Expression
*break}**
default : Expression
}

switch-exp (exp1 exps2 exps3 exp4)

הביטוי החדש מתפקד בדומה לביטוי switch-case-default הקיים בשפת C ובשפות אחרות. כאשר,

- exp1 הוא ביטוי שרוצים לבדוק אם ערכו תואם לאחד מהביטויים של ה-cases השונים.
- exps2+exps3 - הרשימה exps2 היא אוסף ביטויים של ה-cases השונים, במידה וערכו של אחד מהם שווה לערכו של exp1 אזי יוחזר ערכו של הביטוי התואם מ-exps3. אם ישנם מספר ביטויי case תואמים יש להתייחס רק לראשון שבהם.
- exp4 – אם אף אחד מביטויי ה-case לא תואם ל-exp1 או שלא נרשמו בכלל ביטויי case אזי יש להחזיר את ערכו של הביטוי exp4

להלן דוגמא להמחשת השימוש בביטוי החדש :

```
> (run "switch -(5,3) {  
    case 6 : 9  
        break  
    case -(3,9) : 20  
        break  
    case -(8,6) : zero?(8)  
        break  
    case zero? (9) : 200  
        break  
    default: 111  
}  
")  
(bool-val #f)  
>
```

הסבר הדוגמה :

ערכו של ביטוי ה-switch הוא 2 והביטוי ב-case השלישי מתאים לכך, על כן, הוחזר ערכו של הביטוי zero?(8) שהוא (bool-val #f)

א. (7 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי הדרוש בדקדוק של של השפה כך שיתמוך בהגדרת הביטוי החדש.

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% " (arbno (not #\newline)))) skip)
    (identifier (letter (arbno (or letter digit "_" "-" "?")))) symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")") diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression ("if" expression "then" expression "else" expression) if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression) let-exp)
  ))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת "ויהי" כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את הביטוי החדש.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

;;value-of : Exp x Env ----> ExpVal

```
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))
      (var-exp (var) (apply-env env var))
      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val (- num1 num2))))))
      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1) (bool-val #t) (bool-val #f)))))
      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
      (let-exp (var exp1 body)
        (let ((val1 (value-of exp1 env)))
          (value-of body (extend-env var val1 env))))
    )))
```

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

בשאלה זו, ברצוננו לשנות את אופן ההגדרה של פרוצדורה, כך שתתמוך גם בהגדרת משתנים לוקליים (מקומיים).

להלן הדקדוק המעודכן עבור ביטוי הגדרת פרוצדורה proc-exp :

$Expression ::= \text{proc} (identifier)$
 {
 {def identifier = Expression;}^{*}
 Expression
 }

proc-exp (idl ids vals body)

כאשר,

- idl הוא הארגומנט של הפרוצדורה.
- ids – רשימת שמות של המשתנים הלוקליים.
- vals – רשימת ערכים עבור המשתנים הלוקליים.
- Body הוא ביטוי המתאר את גוף הפרוצדורה שתיבנה.

הסבר:

פרוצדורות מוגדרות ומופעלות כפי שהיה בשפת "וישגור", כלומר ע"י Static Scoping בזמן שהפרוצדורה תופעל היא תכיר גם את המשתנים הלוקליים עם הערכים עבורם, בנוסף לארגומנט של הפרוצדורה.

להלן דוגמה להמחשה:

```
let x=10
in
  let y = proc(a)
  {
    def b= 20;
    def x=50;

    -(x,a)
  }
  in
    (y 5)
```

הסבר : בדוגמה זו יוחזר ערכו של הביטוי (y 5) בעקבותיו תופעל הפרוצדורה שהוגדרה עם סביבה המכירה את x כ-10. אך כאשר מופעלת הפרוצדורה מתווספים משתנים לוקליים שאחד מהם הוא גם x עם ערך 50. ולכן הפרוצדורה תשתמש בו.
 התוצאה הסופית של דוגמה זו היא : (num-val 45)

א. (7 נקודות)

להלן נתונים ה-scanner וה-parser של שפת "וישגור". כתבו מהו השינוי הדרוש בדקדוק של השפה כך שיתמוך בהגדרת הביטוי החדש.

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% (arbno (not #\newline))) skip)
    (identifier(letter (arbno (or letter digit "_" "- " "?"))
                     symbol)
      (number (digit (arbno digit)) number)
      (number ("-" digit (arbno digit)) number)
    ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)

    (expression (number) const-exp)
    (expression
      ("-" "(" expression "," expression ")")
      diff-exp)

    (expression
      ("zero?" "(" expression ")")
      zero?-exp)

    (expression
      ("if" expression "then" expression "else" expression)
      if-exp)

    (expression (identifier) var-exp)

    (expression
      ("let" identifier "=" expression "in" expression)
      let-exp)

    (expression
      ("proc" "(" identifier ")" expression)
      proc-exp)

    (expression
      "(" expression expression ")"
      call-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת "וישגור". ולאחריה פרוצדורת העזר *apply-procedure* כפי שמופיעה בספר הלימוד בעמוד 79. כתבו מה העדכון שנדרש להכניס על מנת לתמוך בחישוב של הביטוי החדש.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

;;value-of : Exp x Env→ExpVal

```
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))
      (var-exp (var) (apply-env env var))
      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val (- num1 num2))))))
      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1) (bool-val #t) (bool-val #f)))))
      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))
      (let-exp (var exp1 body)
        (let ((val1 (value-of exp1 env)))
          (value-of body (extend-env var val1 env))))
      (proc-exp (var body)
        (proc-val (procedure var body env)))
      (call-exp (rator rand)
        (let ((proc (expval->proc (value-of rator env)))
              (arg (value-of rand env)))
          (apply-procedure proc arg))))))
;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79
(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body (extend-env var val saved-env))))))
```

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם `ChangeBody`.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

$Expression ::= \text{ChangeBody} (identifier, Expression)$

ChangeBody-exp (prc newbody)

לביטוי **ChangeBody-exp** המרכיבים הבאים:

- **prc** – שם של פרוצדורה שכבר הוגדרה קודם לכן.
- **Newbody** – ביטוי שיחליף את גוף הפרוצדורה שהוגדרה.

ביטוי `ChangeBody` אמור להחליף לפרוצדורה שכבר הוגדרה, את גוף הפרוצדורה בגוף חדש, כל יתר פרטי הפרוצדורה יוותרו ללא שינוי. כך שמעתה אם הפרוצדורה תופעל היא תבצע את גוף הפרוצדורה החדש ביטוי זה אינו מחזיר תשובה, והוא דומה לביטוי השמה המחזיר ערך פיקטיבי.

להלן דוגמא:

```
let dec = proc (x) -(x,1)
in begin
  ChangeBody(dec , -(x,2));
  (dec 8)
end
```

בדוגמה זו, מוגדרת פרוצדורה בשם `dec` המקבלת פרמטר `x` ומחזירה `x-1`.

לאחר מכן, משנים את גוף הפרוצדורה בלבד לביטוי חדש `x-2`.

ולבסוף מופעלת הפרוצדורה על ארגומנט שערכו 8. ולכן ערכה הסופי של התוכנית הוא

(num-val 6)

(המשך השאלה בעמוד הבא)

א. (7 נקודות)

להלן נתונים ה-scanner וה-parser של שפת "וירמוז". כתבו מהו השינוי הדרוש בדקדוק של של השפה כך שיתמוך בהגדרת הביטוי החדש.

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("% (arbno (not #\newline))) skip)
    (identifier(letter (arbno (or letter digit "_" "- " ?"))
      symbol)
      (number (digit (arbno digit)) number)
      (number ("-" digit (arbno digit)) number)
    ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")")
      diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression
      ("if" expression "then" expression "else" expression)
      if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression)
      let-exp)
    (expression ("proc" "(" identifier ")" expression)
      proc-exp)
    (expression "(" expression expression ")") call-exp)
    (expression
      ("letrec"
        (arbno identifier "(" identifier ")" "=" expression)
        "in" expression)
      letrec-exp)
    (expression ("set" identifier "=" expression)
      assign-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו-*apply-procedure* של שפת "וירמוז".
הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
(define value-of
  (lambda (exp env)
    (cases expression exp
      (const-exp (num) (num-val num))

      (var-exp (var) (deref (apply-env env var)))

      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val
             (- num1 num2))))))

      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1)
                (bool-val #t)
                (bool-val #f))))))

      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

      (let-exp (var exp1 body)
        (let ((v1 (value-of exp1 env)))
          (value-of body
                     (extend-env var (newref v1) env))))

      (proc-exp (var body)
        (proc-val (procedure var body env)))

      (call-exp (rator rand)
        (let ((proc (expval->proc (value-of rator env)))
              (arg (value-of rand env)))
```

```

        (apply-procedure proc arg)))

(letrec-exp (p-names b-vars p-bodies letrec-body)
  (value-of letrec-body
    (extend-env-rec* p-names b-vars p-bodies env)))

(assign-exp (var expl)
  (begin
    (setref!
      (apply-env env var)
      (value-of expl env))
    (num-val 27))))))

(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body
          (extend-env var (newref val) saved-env))))))

```

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p = (proc (w:?) -(w,7) 9)
in
if zero? (p) then p else (proc (y:?) -(y,7) 20)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !